Aim
To create and deploy a Ballot smart contract using Solidity and Remix IDE and perform voting and vote delegation transactions.

Theory
1. Smart Contract
A **smart contract** is a program stored on a blockchain that automatically executes predefined rules when its functions are called. In this experiment, a **Ballot smart contract** is used to implement a voting system where voters can receive voting rights, vote for proposals, and delegate their votes.

2. require Statement
The require statement is used to check whether a condition is satisfied before executing a function. If the condition is false, the transaction stops and the changes made during that transaction are reverted.

This ensures that only the chairperson can give voting rights to another voter.
In the Ballot contract, require is also used to:
- Check whether a voter has voting rights.
- Prevent a voter from voting more than once.
- Prevent self-delegation.
- Check whether a voter already has voting rights.

Thus, require helps maintain **security, correctness and reliability** of the smart contract.

3. Mapping
A **mapping** is a data structure that associates a key with a value.
Syntax: mapping(address => Voter) public voters;
In the Ballot contract, each Ethereum address is mapped to its corresponding Voter information.
It stores details such as:
- Voting weight
- Whether the voter has voted
- Delegate address
- Proposal voted for

Mappings are useful for quickly finding voter information using their address.

4. Storage
Storage is the permanent data storage of a smart contract on the blockchain.
Data stored in storage remains available between different function calls and transactions.

5. Memory
Memory is temporary storage used while a function is executing.
For example: constructor(string[] memory proposalNames)

Here, proposalNames is temporarily stored in memory while the constructor executes.
Memory is cheaper than storage because the data is discarded after the function execution is completed.
Blockchain Experiments 6

6. bytes32 and string
The original Ballot contract uses:
bytes32 name;
bytes32 is a fixed-size data type that stores exactly 32 bytes and is useful for short names and gas-efficient operations.
However, it has a limitation on the size of the stored text.

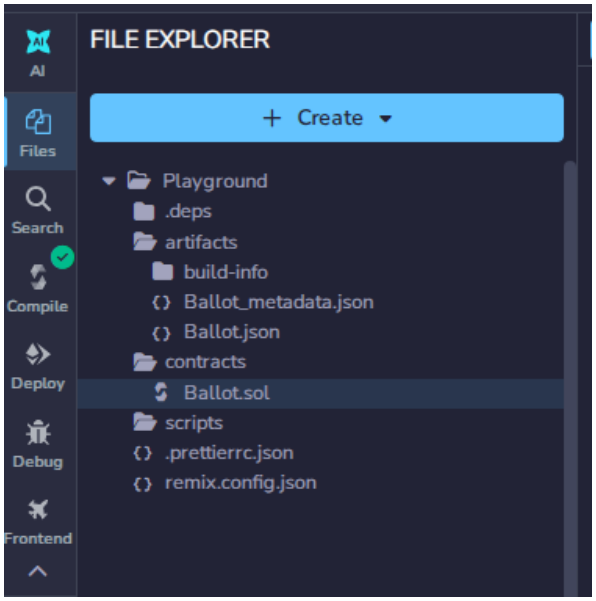
7. Ballot Contract

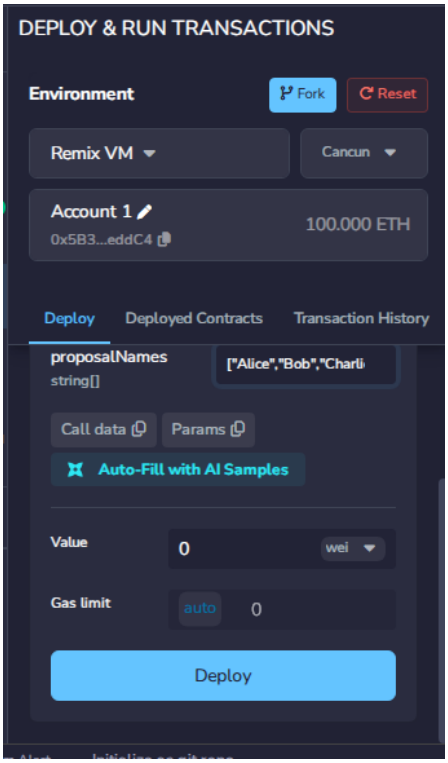
The Ballot contract contains:

- Voter structure to store voter information.
- Proposal structure to store proposal names and vote counts.
- chairperson to identify the account controlling voting rights.
- voters mapping to store voter information.
- proposals array to store proposal information.
- giveRightToVote() to give voting rights.
- delegate() to delegate voting power.
- vote() to cast a vote.
- winningProposal() to find the winning proposal.
- winnerName() to display the winner.

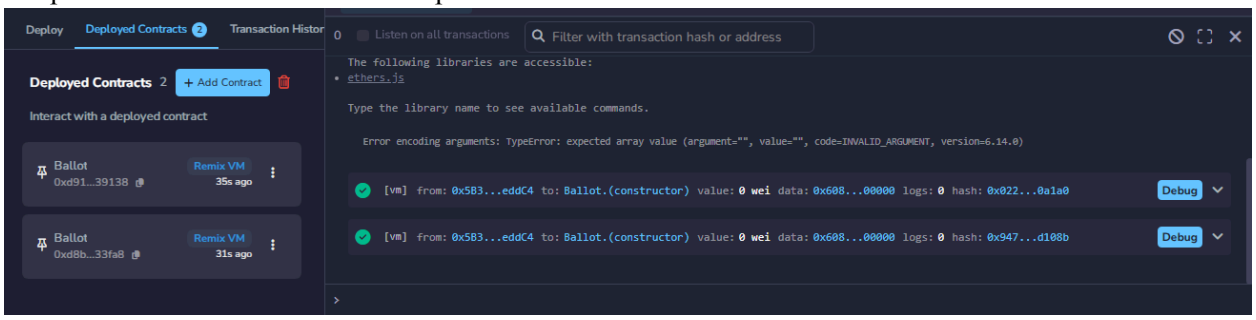
These functions form the basic voting and delegation mechanism of the experiment.

OUTPUT

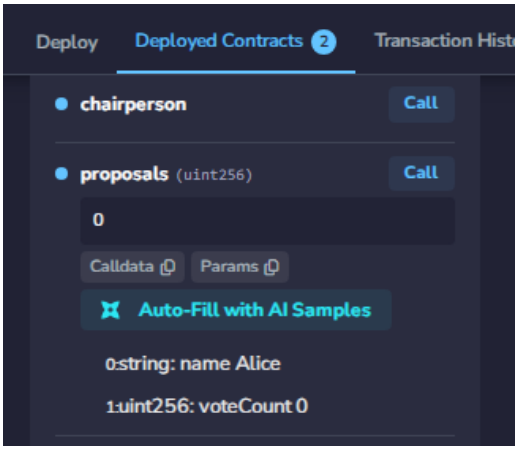


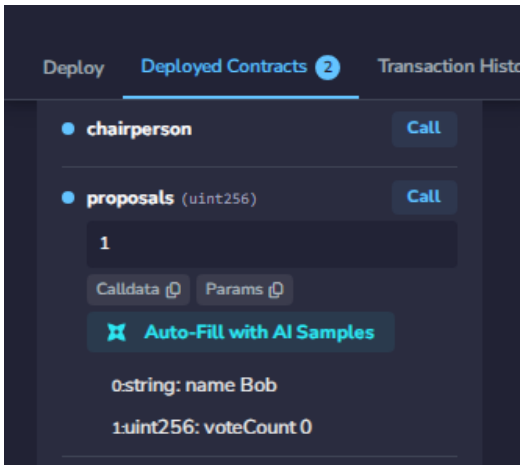


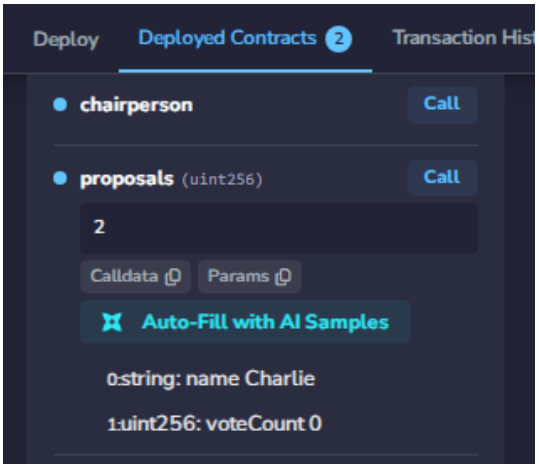
The Ballot smart contract was successfully compiled in Remix IDE without compilation errors.



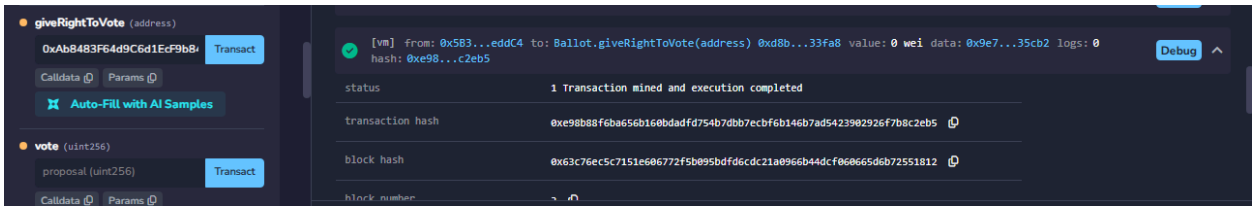
Three proposal candidates, Alice, Bob and Charlie, were successfully loaded into the Ballot contract.



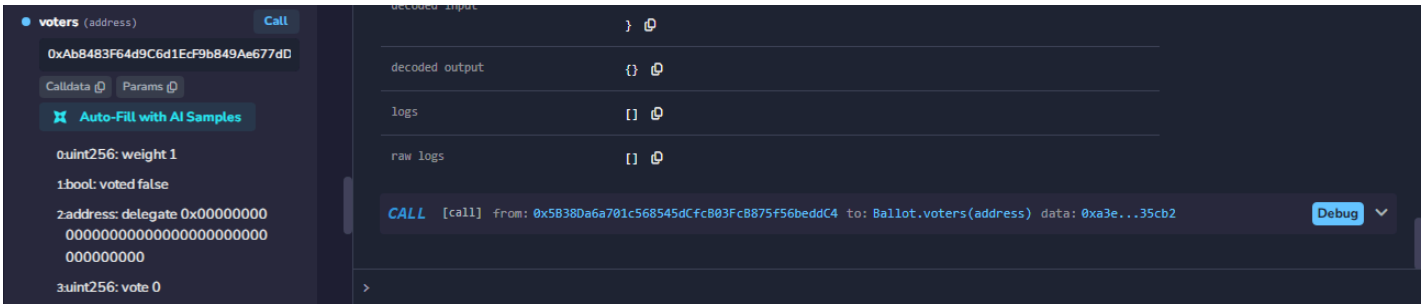


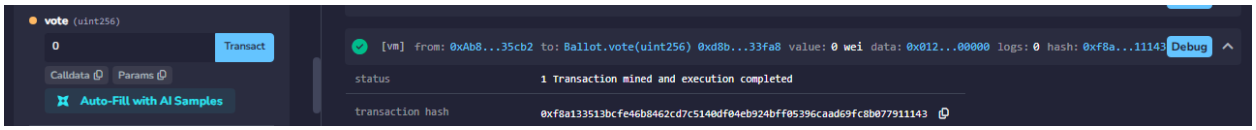


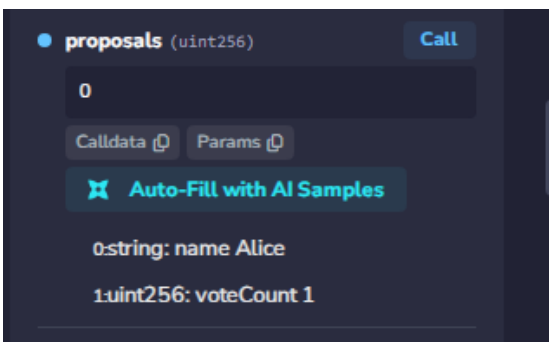
Proposal 0 corresponds to
Alice,proposal 1 corresponds to Bob and proposal 2 corresponds to Charlie.

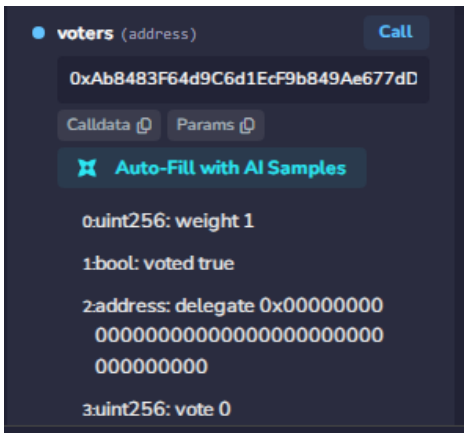


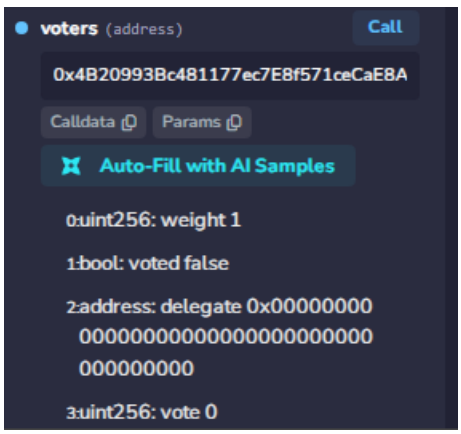
The chairperson successfully granted voting rights to another account.

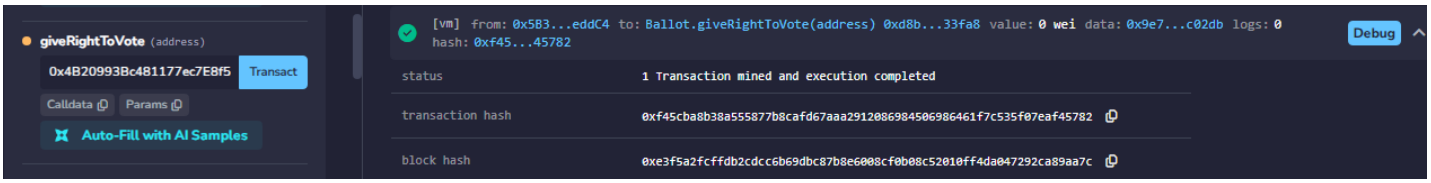




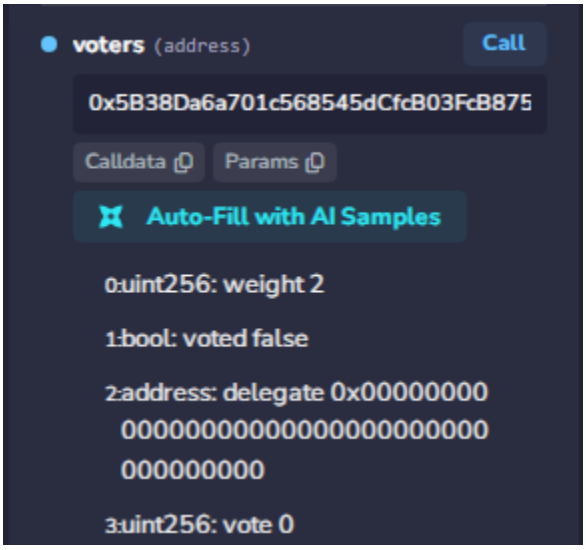


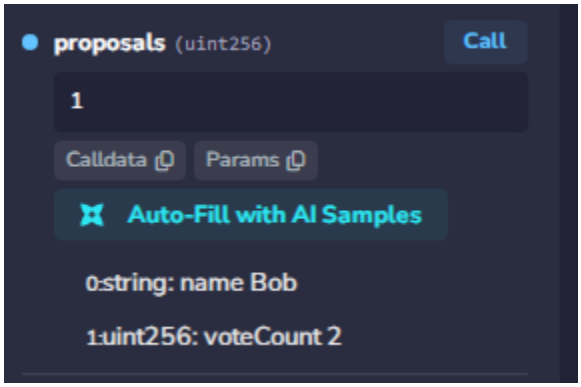


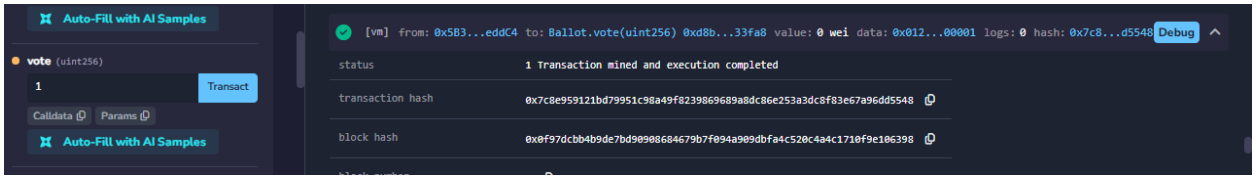


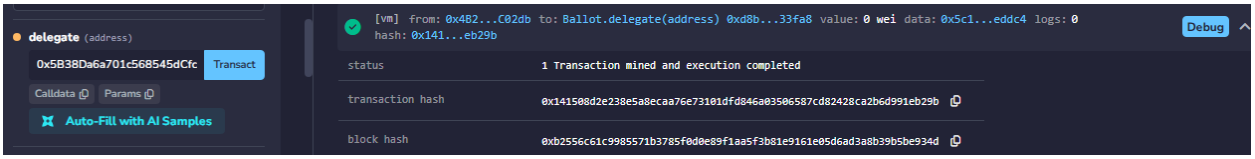


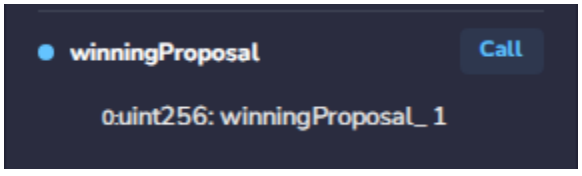
After voting, the voter's status changed to voted = true, confirming that the vote was recorded successfully. Account 3 successfully delegated its voting power to Account 1.











CONCLUSION

Thus, the creation and deployment of a Ballot smart contract using Solidity and Remix IDE was successfully performed. The experiment helped in understanding smart contract deployment, transactions, require statements, mappings, storage and memory, voting, vote delegation, and the difference between bytes32 and string. The successful execution demonstrated how blockchain-based voting can record and manage voting information securely and transparently.